Project 3: Kernel Threads and Kernel-space Shared-memory IPC

Due by **October 11, 2024, 11:59 pm** (**FIRM**. The deadline will not be extended. Fall break starts on Oct 12.)

Summary

In this project, we will use a kernel module to implement kernel threads and use kernel semaphores to synchronize these threads. We will develop it based on the VM and kernel we built in Project 1 and the kernel module we built in Project 2. This project will provide us with interesting kernel development experience and help us understand how to perform synchronization in real-world operating systems.

Description

In this project, you will implement a kernel module that launches multiple kernel threads and implement semaphores in these threads. To make the project interesting and to prepare for the next project, we will implement two types of threads: ***producer*** threads and ***consumer*** threads. Both types of threads work in infinite loops until the module is unloaded.

1. Implement a new module

You **MUST** name your module **"producer_consumer"**.

It takes **three** arguments and you **MUST** name your input parameters for the kernel module as follows such that the test script can load the kernel module successfully. Failure to do so, your kernel module will not be loaded by the automated test script and you will receive **zero** grade points.

a. **prod:** int, number of producer threads (a non-negative number)
b. **cons:** int, number of consumer threads (a non-negative number)
c. **size:** int, initializes the counting semaphore (a non-negative number)

Note that the number of producer/consumer threads can be zero.

2. Create and start kernel threads

Use the `kthread_run()` function in your module to create and start the kernel threads. You will need to start **prod** number of producer threads and **cons** number of consumer threads.

```
/* threadfn is the function to run in the thread;
* data is the data pointer for threadfn; namefmt is the name for the
thread. It returns a pointer to the thread's task_struct if the
```

```
thread creation is successful or ERR_PTR(-ENOMEM) if it fails./
struct task_struct *kthread_run(int (*threadfn)(void *data), void
*data, *const char *namefmt, ...)
```

Example code to create and run a kernel thread:

```
#include <linux/kthread.h>
// the function to run in the thread
static int kthread_func(void *arg) {
    ...
}
// Create a thread named "thread-1" and run it
ts1 = kthread_run(kthread_func, NULL, "thread-1");

// Another way to do this - useful for numbering each thread
ts1 = kthread_run(kthread_func, NULL, "thread-%d", 1);
```

For testing purposes, you **MUST** name your producer and consumer threads as follows:

- "Producer-0", "Producer-1", …, "Producer-n"
- "Consumer-0", "Consumer-1", … "Consumer-m"

3. Implement the critical section

The main focus of this project is to introduce you to semaphores and synchronization between multiple kernel threads. Neither the producer or consumer threads will produce or consume anything real, and instead will simply generate an output as if they were.

The outputs of your producer and consumer threads **MUST** follow the following convention:

- For every producer thread**:**
  - An item has been produced by Producer-<producer-thread-id>
  - Example: **"An item has been produced by Producer-1"**
- For every consumer thread:
  - An item has been produced by Consumer-<consumer-thread-id>
  - Example: **"An item has been produced by Consumer-1"**

To achieve synchronization between your producer and consumer threads, you are to implement two semaphores, initialized as follows:

- **empty**, initialized to the value of **size** (the module parameter)
- **full**, initialized to **0**

The following explains how to use a kernel semaphore. Check the header file semaphore.h for all the semaphore function prototypes.

- Declare your semaphore and initialize it

```
struct semaphore sem;
static inline void sema_init(struct semaphore *sem, int val);
```

`sema_init` is used to initialize a semaphore.
  - @sem: semaphore to initialize
  - val: initializes the count member of semaphore
- Acquire and Release semaphores:
  - `void up(struct semaphore *sem);`
  - `int down_interruptible(struct semaphore *sem);`
    - If no more tasks are allowed to acquire the semaphore, calling this function will put the task to sleep. If the sleep is interrupted by a signal, this function will return **-EINTR**. If the semaphore is successfully acquired, this function returns **0**.

For reference; the code of your producer and consumer threads is as follows:

Producer thread

```
down_interruptible(&empty);
printk("An item has been produced by Producer-%d\n", id);
up(&full);
```

Consumer thread

```
down_interruptible(&full);
printk("An item has been consumed by Consumer-...\n", id);
up(&empty);
```

In addition, from within a thread, you can use the `get_task_comm()` function to retrieve the name of the thread:

```
char thread_name[TASK_COMM_LEN] = { };
get_task_comm(thread_name, t); // t is a task_struct pointer
```

4. Stop the kernel threads

To stop a kernel thread, you need to use `kthread_stop(struct task_struct *ts1)` to signal thread ts1 to stop. The thread can call `kthread_should_stop` to check if it should stop; if it is already signaled to stop, `kthread_should_stop` will return true.

```
// stop the kernel thread pointed by task_struct ts1
kthread_stop(struct task_struct *ts1)

// when kthread_stop() is called, this function will return true
kthread_should_stop(void)
```

When the module unloads, it should terminate **ALL** the producer and consumer threads. To do so, you need to call `kthread_stop` for each producer and consumer thread to stop them.

Now that you know how to stop a thread, you should put the critical section implemented in Step 3 in an infinite loop, but remember to check kthread_should_stop so every thread can stop when the module unloads.

You can use `ps` to check if all of your threads are terminated.

Testing

- In order to test your project we have provided you with the grading scripts. They are available on a GitHub repository: https://github.com/visa-lab/CSE330-OS.git
- In order to clone the Github repository follow the below steps:
  - git clone https://github.com/visa-lab/CSE330-OS.git
  - cd CSE330-OS/
  - git checkout project-3
- Use test_module.sh to test your module code.
  - The script takes a path to your project directory (i.e., where you have your source code), the value for **prod**, the value for **cons**, and the value for **size**.
  - The script will go into your project directory, compile your kernel module, load your kernel module, and verify the correctness of your implementation based on the provided module parameters.

- The script will print a detailed log after testing each part of the rubrics and point out reasons if grade points were deducted.
- Below shows an example of the expected output from Test Case #1:

```
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$ ./test_module.sh ../Project3/ 5 5 5
Testing your kernel module with 5 producers, 5 consumers, and a size of 5:
[log]: Look for Makefile
[log]: - file /home/vboxuser/Project3/Makefile found
[log]: Look for source file (my_name.c)
[log]: - file /home/vboxuser/Project3/producer_consumer.c found
[log]: Compile the kernel module
[log]: - Compiled successfully
[log]: Load the kernel module
[sudo] password for vboxuser:
[log]: - Loaded successfully
[log]: - Found all expected threads
[log]: Checking output
[log]: - Output is correct
[log]: Unload the kernel module
[log]: - Kernel module unloaded sucessfully
[log]: Checking to make sure kthreads are terminated
[log]: - All threads have been stopped
[producer_consumer]: Passed with 20 out of 20
[Total Score]: 20 out of 20
vboxuser@vboxuser-VirtualBox:~/CSE330-OS$
```

- Use test_zip_contents.sh to validate that your zip file adheres to the directory structure that is expected. Below shows an example of the expected output:

```
vboxuser@vboxuser-VirtualBox:~/git/CSE330-OS$
vboxuser@vboxuser-VirtualBox:~/git/CSE330-OS$
vboxuser@vboxuser-VirtualBox:~/git/CSE330-OS$ ./test_zip_contents.sh /tmp/submit/Project-3-1234567890.zip
[log]: Look for source file (my_name.c)
[log]: - file /home/vboxuser/git/CSE330-OS/unzip_1726897369/source_code found
[log]: Look for Makefile
[log]: - file /home/vboxuser/git/CSE330-OS/unzip_1726897369/source_code/Makefile found
[log]: Look for source file (my_name.c)
[log]: - file /home/vboxuser/git/CSE330-OS/unzip_1726897369/source_code/producer_consumer.c found
[test_zip_contents]: Passed
vboxuser@vboxuser-VirtualBox:~/git/CSE330-OS$
```

- Note: We will check your code in addition to running the test script, so passing the testing script does not guarantee getting all the grade points. But if your submission fails to pass the test_zip_contents.sh script, you will receive **zero** grade points.
- Suggestion: You can use "sudo dmesg -C" to clear the dmesg logs after running each test

Submission Requirements & Guidelines

- Submit the following in a zip file to Canvas:
  - Source code: Make a directory named '*source_code*', and include your code file named '*producer_consumer.c*' and your makefile named *'Makefile'*.
- You **MUST** name the zip file following the below naming convention: "Project-3-<Your ASU ID>.zip", e.g., Project-3-1225754101.zip
  - You can use the following command to create your zip file:
    `zip -r Project-3-1225754101.zip source_code/`

  Note: Extensions automatically generated by Canvas due to multiple submission attempts will be managed by the grading script.
- Do not submit any other source code

- Do not submit any binary
- If your submissions do not adhere to submission guidelines and naming conventions you will receive zero points.

Grading Rubrics

Criteria	Test	Points
Test case #1 1. Kernel module should load without errors 2. 5 producer threads and 10 consumer threads are started 3. Each thread prints the required message 4. Kernel module should unload without errors 5. All threads are stopped when the module is unloaded	**Input:** `./test_module.sh /path/to/code/ 5 10 5` **Expected Output:** 1. The test script successfully validates the producer and consumer printouts from dmesg. 2. The test script successfully validates the thread count prod=5 and cons=10. 3. The script is able to successfully load and unload your kernel module. 4. The script successfully validates that all kernel threads have been stopped after unloading your kernel module.	2 pts load module 8 pts dmesg logs 4 pts correct thread count 2 pts unload module 4 pts all threads stopped --- 20 points total
Test case #2 1. Kernel module should load without errors 2. 5 producer threads and 0 consumer threads are started	**Input:** `./test_module.sh /path/to/code/ 5 0 5` **Expected Output:** 1. The test script successfully validates the producer and consumer printouts from dmesg. 2. The test script successfully validates the thread count prod=5 and cons=0.	2 pts load module 8 pts dmesg logs 4 pts correct thread count 2 pts unload module 4 pts all threads stopped --- 20 points total

3. Each thread prints the required message 4. Kernel module should unload without errors. 5. All threads are stopped when the module is unloaded	3. The script is able to successfully load and unload your kernel module. 4. The script successfully validates that all kernel threads have been stopped after unloading your kernel module.	
Test case #3 1. Kernel module should load without errors 2. 0 producer threads and 5 consumer threads are started 3. Each thread prints the required message 4. Kernel module should unload without errors. 5. All threads are stopped when the module is unloaded	**Input:** `./test_module.sh /path/to/code/ 0 5 5` **Expected Output:** 1. The test script successfully validates the producer and consumer printouts from dmesg. 2. The test script successfully validates the thread count prod=0 and cons=5. 3. The script is able to successfully load and unload your kernel module. 4. The script successfully validates that all kernel threads have been stopped after unloading your kernel module.	2 pts load module 8 pts dmesg logs 4 pts correct thread count 2 pts unload module 4 pts all threads stopped --- 20 points total

Test case #4 1. Kernel module should load without errors 2. 0 producer threads and 5 consumer threads are started 3. Each thread prints the required message 4. Kernel module should unload without errors. 5. All threads are stopped when the module is unloaded	**Input:** `./test_module.sh /path/to/code/ 5 5 0` **Expected Output:** 1. The test script successfully validates the producer and consumer printouts from dmesg. 2. The test script successfully validates the thread count prod=5 and cons=5. 3. The script is able to successfully load and unload your kernel module. 4. The script successfully validates that all kernel threads have been stopped after unloading your kernel module.	2 pts load module 8 pts dmesg logs 4 pts correct thread count 2 pts unload module 4 pts all threads stopped --- 20 points total
Test case #5 1. Kernel module should load without errors 2. 50 producer threads and 50 consumer threads are started 3. Each thread prints the required message 4. Kernel module should unload without errors. 5. All threads are stopped when the module is unloaded	**Input:** `./test_module.sh /path/to/code/ 50 50 10` **Expected Output:** 1. The test script successfully validates the producer and consumer printouts from dmesg 2. The test script successfully validates the thread count prod=50 and cons=50. 3. The script is able to successfully load and unload your kernel module. 4. The script successfully validates that all kernel threads have been stopped after unloading your kernel module.	2 pts load module 8 pts dmesg logs 4 pts correct thread count 2 pts unload module 4 pts all threads stopped --- 20 points total

Submission Content	`./test_zip_contents.sh /path/to/zip/file.zip` 1. Zip file should contain only raw outputs (screenshot), source code. No binaries. 2. All files adhere to the directory structure specified in the project spec. Failure to adhere to the directory structure will result in zero.	-5 pts unwanted files or binaries

Policies

- Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
- Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.
- **The use of generative AI tools is not allowed** to complete any portion of the assignment.